

Claude Code — Session Management Guide

Save, Rename & Resume Chats/Sessions

Source: Official Claude Code Docs (code.claude.com/docs/en/common-workflows)

Last updated: April 2026 | Claude Code v2.1.x

How Sessions Work

All conversations in Claude Code are **automatically saved locally** — no manual action needed. When you resume, the entire session is restored:

- Complete message history
- Tool usage and results
- Model and configuration
- Working directory context

Storage location:

```
~/.claude/projects/<encoded-project-path>/*.jsonl
```

Each `.jsonl` file is a complete conversation. Sessions **never auto-delete**.

Starting a Named Session

The single best habit — name a session **when you start it**, not after.

```
# Short form
claude -n "feature-auth-system"

# Long form
claude --name "bugfix-stripe-webhook"
```

Naming Conventions That Work

```
# ✅ Good names
claude -n "feature-user-auth"
claude -n "bugfix-payment-retry"
claude -n "refactor-api-routes"
claude -n "review-pr-142"
claude -n "docs-api-reference"

# ❌ Avoid these
claude -n "work"           # Too vague
claude -n "test"          # What test?
claude -n "2026-04-15"    # Date alone doesn't convey purpose
```

Tip: Consistent prefixes like `feature-`, `bugfix-`, `refactor-`, `docs-`, `review-` make your session list scannable at a glance.

Renaming a Session

During an Active Session

```
/rename payment-integration      # give a specific name
/rename                          # no argument = auto-generates from conversation history
```

Auto-generating uses a few extra tokens but saves you thinking of a name yourself.

From the Session Picker

```
/resume      # open the picker
              # navigate to any session
              # press R to rename it
```

Resuming Sessions — All Methods

1. Continue Last Session (Quickest)

Picks up exactly where you left off in the most recent session for the current directory. No picker, no selecting.

```
claude --continue
claude -c      # short form
```

Best for: "I stepped away for lunch and need to continue."

2. Browse All Sessions (Interactive Picker)

Opens an interactive list of all your sessions.

```
claude --resume
claude -r      # short form
```

Keyboard shortcuts inside the picker:

Key	Action
Enter	Resume selected session
P	Preview session before resuming
R	Rename the selected session
A	Toggle between current directory and all projects
B	Filter by current git branch

3. Resume a Specific Session by Name or ID

```
# By name (if you used /rename or --name)
claude --resume "feature-auth-system"
claude -r "bugfix-stripe-webhook"
```

```
# By session ID
claude --resume abc123def
```

4. Switch Sessions Mid-Chat (Without Exiting)

You don't need to exit and relaunch. From inside any active session:

```
/resume
```

The same picker opens. Select a session and switch instantly.

This is huge if you bounce between tasks a lot — no terminal juggling needed.

5. Resume a Session Linked to a Pull Request

When you create a PR with `gh pr create`, Claude automatically links the session to that PR.

```
claude --from-pr 142
```

6. Resume + Send a Prompt in One Step (Scripting / Headless)

```
claude --continue --print "continue with the frontend implementation"
claude -c -p "run the tests and fix any failures"
```

Exporting a Session to a File

Save the conversation as plain text — useful for documentation, sharing with teammates, or keeping a record before a risky refactor.

```
/export # opens dialog: copy to clipboard or save to file
/export payments-context.md # saves directly to that filename
```

Forking a Session (Branching)

When you want to try a different direction without losing your current thread.

```
/branch # creates a fork at this point in the conversation
/fork # alias
```

Or from the CLI at startup:

```
claude --fork-session <session-id>
```

The fork gets its own session ID. The original is untouched. You can resume both independently.

Use case: "I've analysed the auth module. Now I want to explore OAuth2 in one branch and JWT in another — without losing either thread."

Managing Context in Long Sessions

Sessions grow as you work. Once the context window fills up, quality can degrade.

Check Context Usage

```
/context
```

Shows a visual grid of how full your context is, plus optimization suggestions.

Compact (Summarise Older Messages)

```
/compact                                # summarise everything older
/compact keep the database design decisions  # guided compaction
/compact keep architecture choices, summarise the rest
```

Consider compacting when:

- Token usage exceeds 50%
- You're switching to a different topic
- After extensive trial-and-error (failed approaches can be summarised away)

Clear and Start Fresh

```
/clear      # wipe and start fresh
/reset      # alias
/new        # alias
```

After completing a feature, use `/clear` to reset context. Each session focused on a single topic is easier to resume and less prone to hallucination from overloaded context.

Day-to-Day Workflow Pattern

```
# Monday morning – start a named session for a specific task
claude -n "feature-payments-v2"

# ... work all day ...
# Close terminal – session is auto-saved

# Tuesday morning – pick up exactly where you left off
claude --continue

# Later – need to switch tasks mid-day
/resume                # open picker, select "bugfix-login-redirect"
```

```
# Switch back to payments
/resume feature-payments-v2

# Before a big risky refactor – export a record
/export payments-context-backup.md

# Context getting full – compact but keep key decisions
/compact keep architecture decisions and API design choices

# Feature complete – start clean for the next task
/clear
```

Parallel Task Workflow

Working on multiple features simultaneously:

```
# Terminal window 1
claude -n "feature-auth-oauth"

# Terminal window 2
claude -n "bugfix-report-export"

# Terminal window 3
claude -n "refactor-user-service"

# Switch between them without exiting any
/resume # pick whichever session you need
```

Quick Reference Card

What You Want to Do	Command
Start a named session	claude -n "feature-name"
Rename current session	/rename new-name
Auto-name from history	/rename (no argument)
Continue last session	claude -c OR claude --continue
Browse all sessions	claude -r OR claude --resume
Resume specific session	claude -r "session-name"
Switch sessions mid-chat	/resume
Resume from PR	claude --from-pr 142
Resume + send prompt	claude -c -p "prompt text"
Fork current session	/branch OR /fork

Export conversation	<code>/export [filename]</code>
Check context usage	<code>/context</code>
Compact conversation	<code>/compact [instructions]</code>
Clear and start fresh	<code>/clear</code>

Pro Tips

Name sessions early. It's much easier to find `"payment-integration"` than `"explain this function"` three days later.

One session per task. Don't pile unrelated work into one session. Use `/clear` between distinct tasks and name each one.

Preview before resuming. In the picker, press `P` to preview a session's content before you open it.

Rename from the picker. If you forgot to name a session when starting, press `R` inside the `/resume` picker to rename it retroactively.

Export before big refactors. Use `/export` to create a paper trail of reasoning before you make drastic changes.

Watch your context. Run `/context` regularly during long sessions. Once it fills up, response quality drops — compact early rather than late.

Source: [Claude Code Common Workflows](#) — Official Anthropic Documentation